Document no: P2114R0
Date: 2020-03-02
Authors: Joshua Berne Ryan McDougall Andrzej Krzemieński
Reply-to: jberne4@bloomberg.net
Audience: SG21

Minimal Contract Use Cases

Introduction

SG21 has been reviewing a large number of potential use cases for contracts. There are also a number of potential proposals that have previously been put forward with might be applicapable to those use cases (see P2032).

We (the authors of this paper, acting inependently) wish to highlight those use cases that mght be viewed as the true common core of the proposals so far. It is our hope to get agreement on this so that the core of a proposal can be started, and so we can address other features and satisfying more use cases iteratively on top of that central proposal (instead of needing to start each proposal from the ground up, and being in a situation of regularly retreading ground while comparing apples to oranges.)

An additional hope is that, as we get proposals that meet this minimal set of use cases, a "minimal viable product" emerges that we might be able to provide for C++23 should a more full-featured proposal not seem to be reachinng consensus in time. We are not proposing any particular solution here, just a potential guideline for what use cases a minimal feature needs to satisfy.

Use Cases

Here we will list those use cases that we think should be included, with their original high-level description (see P1995 for all of the use cases).

#	Code	As A	In Order To	I Want To
1	dev.reason.knowl	Developer	Reason explicitly	Annotate my program anywhere in the code with my current understanding of its structure or execution
If you could take this out, you still wouldn't be providing anything useful to users.				
2	dev.reason.behavior	Developer	Reason about executions	Have annotations affect the execution of my program in accordance with my expectations
3	dev.reason.sideeffects	Developer	Reason about executions	Ensure annotations do not substantially change the meaning of my program whether enabled or disabled
it is important that there is an option to enable contracts and an option to disable contracts that does not include undefined behavior.				
4	dev.readable.syntax	Developer	Have readable annotations	Have annotations with a succinct and elegant syntax

5	dev.readable.keywords	Developer	Have readable annotations	Have annotation keywords or names with intuitive, clear, and unambiguous meanings
6	dev.parsable	Developer	Interoperate with tools or persons	A syntax that can both be parsed and can be reasoned about semantically
7	dev.tooling	Developer	Interoperate with tools or persons	Expose annotations to tools that might leverage them (eg. code linter, static analyzer, semantic prover, compiler sanitizer, binary analyzer, code reviewer, etc.)
8	cppdev.syntax.familiar	C++ Developer	Get up to speed	Have annotations use familiar syntax
9	cppdev.syntax.cpp	C++ Developer	Get up to speed	Have annotations use C++ syntax
10	cppdev.syntax.reuse	C++ Developer	Reuse code	Have annotations use my custom types or functions
11	cppdev.location	C++ Developer	Have a single source of truth	Use same source file for both code and annotations
12	api.communicate.inputsoutputs	API Developer	Communicate my interface to users	Document the expected inputs and expected outputs on my interface
13	api.establish.check	API Developer	Establish a contract	Have validation inform me which output values are unexpected or invalid
14	api.establish.values	API Developer	Establish a contract	Have validation inform user which input values are unexpected or invalid
15	api.establish.preconditions	API Developer	Establish a contract	Have contracts specify their pre-conditions as logical predicates
16	api.establish.postconditions	API Developer	Establish a contract	Have contracts specify their post-conditions as logical predicates
17	api.express.values	API Developer	Express predicates	Make reference to either the values of my inputs, or other in-scope identifiers
18	api.contract.errorhandling	API Developer	Move contract violation out of error handling	Replace uses of error handling to express contract violation (eg. *operator[](size_t n) noexcept [[pre: n < size()]]* instead of throwing)
19	int.conform.violation	Integration Developer	Conform to a contract	Be informed any time an interface's contract is violated
20	int.conform.postconditions	Integration Developer	Conform to a contract	Verify results from a call are expected output values
21	int.build.headeronly	Integration Developer	Build multiple libraries	Use contract-enabled header-only libraries

22	int.build.binaries	Integration Developer	Build multiple libraries	Use contract-enabled binary libraries
23	int.violations.information	Integration Developer	Correct failed checks	Be informed what check failed, when, where, and how
24	int.violations.common	Integration Developer	Unify violation handling	Be able to override how library violations are handled in the combined software to point into my handling code
25	int.control.build	Integrated Software Provider	Ensure the combined software is correct	At build time, turn on and off what checking happens.
26	int.runtime.unchecked	Integrated Software Provider	Test final deliverable	Turn off run time checking to remove checking overhead
27	cpplib.headeronly	C++ Library Developer	Use templates	Be able to ship header only library
28	arch.nomacros	Technical Architect	Maintain quality of code base	Express assertions in a way that does not rely on C macros (i.e., there is no valid technical reason for a programmer not to use the new way, including space, time, tooling, and usability/complexity reasons, compared to C's assert macro)
29	arch.complete	Technical Architect	Have a consistent and holistic contracts facility	Specify preconditions/postconditions/assertions/invariants that express my expectations about the expected valid state of my program in the form of compilable boolean expressions, that can be checked statically or dynamically (as opposed to disjointed state where these features are factored into bits)
30	sdev.bestpractices	Senior Developer	Set an example	Demonstrate best practice in defensive programming
31	sdev.quality	Senior Developer	Enforce code quality	Discourage reliance on observable out-of-contract behavior by causing check failure to hard stop program or build
32	jdev.understand.contracts	Junior Developer	Understand the API	A uniform, fluent description of expected input values, expected output values, side effects, and all logical pre and post conditions
33	jdev.understand.violations	Junior Developer	Understand the API	Be informed when my usage is out of contract
34	jdev.understand.aborting	Junior	Understand	Know why my software is aborting

		Developer	the program	
35	jdev.understand.buildviolation	Junior Developer	Understand the program	Know that my program or build was halted due to contract violation
36	jdev.understand.all	Junior Developer	Understand the facility	Be able to build a program with contracts after reasonably short tutorial
37	jdev.understand.keywords	Junior Developer	Understand the facility	Have keywords with precise and unambiguous meanings
38	jdev.bestpractices	Junior Developer	Improve my code	Learn about software best practices by example
39	adev.fast	Agile Developer	Iterate quickly	Be able to write and modify contracts quickly without heavy boiler plate or up front cost
40	qdev.checkall	Quality Sensitive Developer	Enable full checking	Ensure all checks (pre, post, assert, invariant) are enabled
41	teach.bestpractices	Teacher	Demonstrate best practice	Be able to express defensive programming, programming by contract, and test driven development to introductory students
42	teach.standardized	Teacher	Demonstrate best practice	Not rely on custom libraries or proprietary extensions
43	teach.portable	Teacher	Manage many students	Have examples compilable by a standard compiler on any system
44	teach.teachable	Teacher	Build layers of understanding	Have simple explanation of assertions and their use to support simple programming tasks, including debugging erroneous programs.
45	compiler.best	Compiler Developer	Deliver the best implementation	Have a clear and simple specification that meets clear need

Conclusion

This began as an excercise amongst the authors, and we hope that we have identified something that would be minimally viable to agree on as a group.

We encourage anyone to compare these results with the use cases and convince themselves this is minimally viable for their own users, or provide feedback on what else is needed/what is excessive to increase this initial step on the road to consensus.